Day 80: The Transformer Decoder Block, and Masked Self-Attention

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 12, Day 4 | Complete study guide

Watch: Transformers, Part 7: Decoder (2), masked self-attention (Lennart Svensson) | Masking the future in self-attention (Herman Kamper)

Script: `day80_decoder_block.py` · every number in this guide is copied by program from `day80_run_output.txt`, the real output of that script.

A decoder writes one token at a time, so at each position it may use only what is already written. The causal mask enforces that inside self-attention. Today the decoder block is built (masked self-attention, cross-attention to an encoder, a feed-forward network) and a next-word model is trained on Week 9's real sentences with and without the mask. Without it, the model scores 0.5133 when given whole sentences, as in training, because it reads the very word it should predict (0.536 of its attention goes one position ahead). Asked to predict from the past only, as when writing text, it falls to 0.0877, below always guessing the commonest token (0.1853). The masked model scores the same both ways.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

A **causal mask** sets the attention score of every later key to minus infinity, so position i reads only positions 0..i. A **decoder block** has three add & norm parts: masked self-attention over what has been written, **cross-attention** (queries from the decoder, keys and values from the encoder's output), and Day 78's feed-forward network. A **decoder-only** model, like today's next-word model, leaves out the cross-attention.

REAL-WORLD SCENARIO

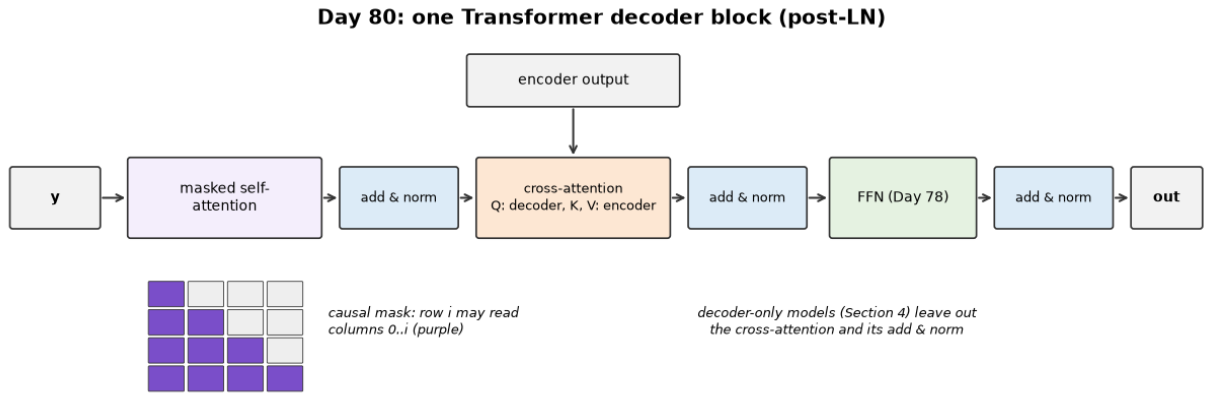
Taking an exam where the answers are printed on the next page: a student who peeks scores well in practice but fails the real test, where the next page is blank.

MATHEMATICS

$$scores[i, j] = -\infty \text{ for } j > i \;\Rightarrow\; weight[i, j] = 0$$

$$h_1 = LN(y + MaskedSelfAttn(y)),\quad h_2 = LN(h_1 + CrossAttn(h_1, enc)),\quad out = LN(h_2 + FFN(h_2))$$

SYSTEM DIAGRAM



One post-LN decoder block, and the causal mask.

Today's part for Day 83

`day80_exercise.py` is causal self-attention and cross-attention on their own, gradient-checked: Week 12's fourth part.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. The causal mask

DEFINITION — start here, no prior knowledge assumed

Day 75's attention with a lower-triangular mask: row i keeps columns 0..i.

REAL-WORLD SCENARIO

A row of people each allowed to look only at those to their left.

IMPLEMENTATION

```
def causal_mask(n):
    return np.tril(np.ones((n, n), dtype=bool))
```

ACTUAL OUTPUT

```
mask (1 = may read):
  1 0 0 0 0
  1 1 0 0 0
  1 1 1 0 0
  1 1 1 1 0
  1 1 1 1 1
weights:
  1.000 0.000 0.000 0.000 0.000   sum 1.000
  0.006 0.994 0.000 0.000 0.000   sum 1.000
  0.265 0.189 0.545 0.000 0.000   sum 1.000
  0.079 0.400 0.407 0.113 0.000   sum 1.000
  0.018 0.556 0.190 0.045 0.191   sum 1.000
largest weight on a future position: 0.0e+00
Actual output from this lesson's run.
```

What it means

Every weight above the diagonal is exactly zero (0.0e+00) and every row still sums to 1.000: the softmax spreads each position's attention over the positions it is allowed to see. Position 0 can only read itself.

WHY CHOOSE THIS?

One line of code enforces 'no peeking' exactly.

WHY NOT THIS? (TRADEOFFS)

The early positions have very little to read.

2. The future cannot leak in

DEFINITION — start here, no prior knowledge assumed

Change one later word and compare every output before and after.

REAL-WORLD SCENARIO

Changing the last line of a letter cannot change what the first line said.

SPECIFICATION

Data	One untrained decoder block, 4 random 12-word sequences; word 8 replaced.

Why this test

It checks the mask through the whole block, not just in one attention matrix.

IMPLEMENTATION

```
def forward(self, y, pe, ymask, enc=None, emask=None):
    P, h, c = self.P, self.h, {}
    m, n = ymask.shape
    allowed = np.broadcast_to(ymask[:, None, :], (m, n, n))
    if self.causal:
        allowed = allowed & causal_mask(n)[None]
    sm = np.repeat(allowed[:, None], h, axis=1).reshape(m * h, n, n)
    Xqk = y + pe
    a, c["self"] = attend(Xqk, Xqk, y, P, "s", h, sm)
    h1, c["n1"] = layer_norm(y + a, P["g1"], P["b1"])
    if self.cross:
        cm = np.repeat(np.broadcast_to(emask[:, None, None, :], (m, 1, n, emask.shape[1])), h,
            axis=1).reshape(m * h, n, -1)
        ca, c["cross"] = attend(h1, enc, enc, P, "c", h, cm)
        h2, c["n2"] = layer_norm(h1 + ca, P["g2"], P["b2"])
    else:
        h2 = h1
    f, c["ffn"] = ffn(h2, dict(W1=P["W1"], c1=P["c1"], W2=P["W2"], c2=P["c2"]))
    out, c["n3"] = layer_norm(h2 + f, P["g3"], P["b3"])
    self.c = c
    return out
```

ACTUAL OUTPUT

```
model              positions 0-7  positions 8-11
causal mask             0.00e+00        5.69e+00
no mask                 1.61e+00        5.72e+00
Actual output from this lesson's run.
```

What it means

With the mask, outputs 0–7 do not move at all (0.00e+00); without it they change by up to 1.61e+00. Outputs from position 8 on change either way, as they should.

WHY CHOOSE THIS?

An exact guarantee, checked end to end.

WHY NOT THIS? (TRADEOFFS)

Untrained weights; the guarantee does not depend on them.

3. Gradient check

DEFINITION — start here, no prior knowledge assumed

The full decoder block with cross-attention, including the gradient for the encoder's output, and the next-word model with and without the mask.

REAL-WORLD SCENARIO

Testing the whole new floor of the building, including the stairs down to the floor below.

MATHEMATICS

$$\text{cross-attention: } Q = h_1 W_q,\ K = enc\ W_k,\ V = enc\ W_v$$

$$d\ enc = dK\ W_k^T + dV\ W_v^T \quad \text{(the encoder made the keys AND the values)}$$

SPECIFICATION

Data	d = 8, 2 heads, FFN 12; 4 decoder positions reading 6 encoder positions, one sequence padded on both sides; the norms' g and b moved off 1 and 0 so they are tested; Day 62's pass rule.

IMPLEMENTATION

```
def attend(Xq, Xk, Xv, P, prefix, h, mask):
    Q = split_heads(Xq @ P[prefix + "q"], h)
    K = split_heads(Xk @ P[prefix + "k"], h)
    V = split_heads(Xv @ P[prefix + "v"], h)
    Zh, A = scaled_dot_product_attention(Q, K, V, mask)
    Z = merge_heads(Zh, h)
    return Z @ P[prefix + "o"], dict(Q=Q, K=K, V=V, A=A, Z=Z, Xq=Xq, Xk=Xk, Xv=Xv)
def attend_backward(dO, P, prefix, h, c):
    flat = lambda T: T.reshape(-1, T.shape[-1])
    g = {prefix + "o": flat(c["Z"]).T @ flat(dO)}
    dQ, dK, dV = scaled_dot_product_attention_backward(split_heads(dO @ P[prefix + "o"].T, h),
        c["Q"], c["K"], c["V"], c["A"])
    dQ, dK, dV = merge_heads(dQ, h), merge_heads(dK, h), merge_heads(dV, h)
    g[prefix + "q"], g[prefix + "k"], g[prefix + "v"] = flat(c["Xq"]).T @ flat(dQ),
        flat(c["Xk"]).T @ flat(dK), flat(c["Xv"]).T @ flat(dV)
    return dQ @ P[prefix + "q"].T, dK @ P[prefix + "k"].T, dV @ P[prefix + "v"].T, g
```

ACTUAL OUTPUT

```
decoder block (self + cross + FFN): largest |numeric - analytic| = 4.79e-10; every entry passes:
    True
next-word model, causal mask : largest |numeric - analytic| = 8.94e-08; every entry passes: True
next-word model, no mask     : largest |numeric - analytic| = 4.75e-08; every entry passes: True
Actual output from this lesson's run.
```

What it means

Every entry passes, for the encoder output too: the gradient that will train an encoder through the decoder (Days 81–83) is right.

WHY CHOOSE THIS?

The decoder's three parts are checked together.

WHY NOT THIS? (TRADEOFFS)

Tiny sizes; the trained model runs the same code.

4a. What is a good score?

DEFINITION — start here, no prior knowledge assumed

Two counting baselines for predicting the next word, with no model at all.

REAL-WORLD SCENARIO

Before praising a forecaster, check how often 'same as yesterday' would have been right.

ACTUAL OUTPUT

```
most common target in training: '<unk>' (words outside the vocabulary); share of test targets that
    are '<unk>': 0.1853
always predicting it: 0.1853      bigram table: 0.2519
Actual output from this lesson's run.
```

What it means

The commonest target is '<unk>', the token for words outside the training vocabulary; it is 0.1853 of the test targets, so always predicting it already scores that. A table of which word most often followed each word in training scores 0.2519.

WHY CHOOSE THIS?

Gives the trained models a fair bar.

WHY NOT THIS? (TRADEOFFS)

Accuracy rewards predicting '<unk>', which says nothing useful.

4. Why the mask: a next-word model with and without it

DEFINITION — start here, no prior knowledge assumed

Input [BOS] w_1 .. w_n, target w_1 .. w_n [EOS]: at position i the model must predict the word after it. Without the mask, position i can read position i + 1, whose input is exactly that word.

REAL-WORLD SCENARIO

The exam with the answers on the next page, practised with the page visible and then sat without it.

SPECIFICATION

Model	One decoder block, no encoder, size 32, 4 heads, FFN 128, Day 77's encoding.
Training	SGD lr 2.0 (Day 78's), clip 1.0, batches of 32, 20 epochs; best epoch by validation cross-entropy predicting from the past only; three seeds.
Two scores	'whole sentence': one pass over the full input, as in training. 'past only': each word predicted from the words before it, as when generating text.

Why this test

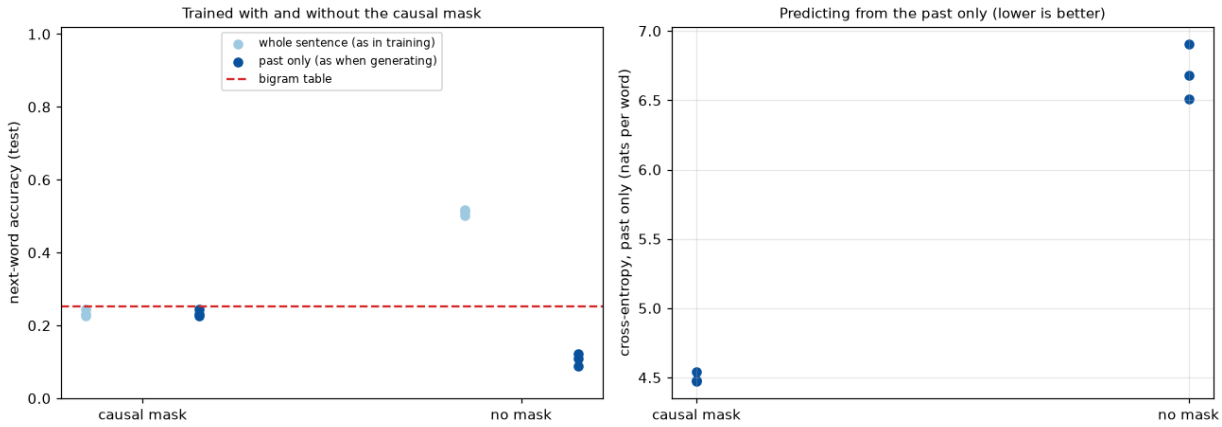
It shows what the mask is for, on real text: the difference between how the model is trained and how it is used.

IMPLEMENTATION

```
def evaluate(net, split, honest):
    x, y, mask = split
    right, nll, count = 0, 0.0, 0
    if not honest:
        for s in range(0, len(x), 32):
            xb, yb, mb = x[s:s + 32], y[s:s + 32], mask[s:s + 32]
            T = mb.sum(axis=1).max(); xb, yb, mb = xb[:, :T], yb[:, :T], mb[:, :T]
            lp = net.forward(xb, xb != PAD).reshape(len(xb), T, -1)
            pick = np.take_along_axis(lp, yb[..., None], axis=-1)[..., 0]
            right += ((lp.argmax(axis=-1) == yb) & mb).sum(); nll -= pick[mb].sum(); count +=
                mb.sum()
        return right / count, nll / count
```

ACTUAL OUTPUT

```
model          seed  whole sentence: acc     CE  past only: acc     CE  best epoch
causal mask       0               0.2254  4.542          0.2254  4.542           7
causal mask       1               0.2442  4.481          0.2442  4.481           6
causal mask       2               0.2326  4.472          0.2326  4.472           6
no mask           0               0.5133  3.283          0.0877  6.680           1
no mask           1               0.5167  3.409          0.1222  6.508           1
no mask           2               0.5007  3.338          0.1102  6.908           1
(CE: mean cross-entropy in nats per word; lower is better)
Actual output from this lesson's run.
```



Left: next-word accuracy on the test set, whole sentence (light) and past only (dark), three seeds; the dashed line is the bigram table. Right: cross-entropy predicting from the past only.

What it means

Without the mask the model looks good and is useless. Given whole sentences it scores 0.5133, 0.5167 and 0.5007; predicting from the past only, 0.0877, 0.1222 and 0.1102, below always guessing '<unk>'. Its best epoch by honest validation is the first: the more it trains, the more it relies on peeking. (Its whole-sentence score would likely be higher at a later epoch; the snapshot kept is the one that predicts best from the past.)

With the mask both scores are the same (0.2254, 0.2442, 0.2326), because there is nothing to peek at. That is still a little below the bigram table: one small block trained with SGD on 2,000 sentences does not beat counting word pairs on accuracy here.

WHY CHOOSE THIS?

Shows the cost of a missing mask in the only score that matters.

WHY NOT THIS? (TRADEOFFS)

One small model on a small corpus; a larger model would be needed to beat the bigram table.

5. With the mask, one pass equals one pass per word

DEFINITION — start here, no prior knowledge assumed

For the trained causal model: the predictions from one pass over whole sentences, against feeding every prefix separately.

REAL-WORLD SCENARIO

Marking a whole exam at once or question by question gives the same marks, if no answer depends on a later page.

Why this test

This equality is why masked decoders can be trained on whole sentences in one pass, cheaply.

ACTUAL OUTPUT

```
largest difference over every predicted position: 7.11e-15
Actual output from this lesson's run.
```

What it means

They agree to rounding (7.11e-15): training on the whole sentence teaches exactly the model that will be used word by word.

WHY CHOOSE THIS?

Training is one pass per sentence, not one per word.

WHY NOT THIS? (TRADEOFFS)

Generation still needs one pass per new word.

6. Where the unmasked model looks

DEFINITION — start here, no prior knowledge assumed

The mean attention weight from each position to the next position, to itself and to the previous one.

REAL-WORLD SCENARIO

Watching the student's eyes during the practice exam.

ACTUAL OUTPUT

```
model           one ahead   itself  one behind
causal mask         0.000    0.349       0.197
no mask             0.536    0.071       0.013
Actual output from this lesson's run.
```

What it means

The unmasked model puts 0.536 of its attention one position ahead, where the answer sits; the masked model cannot (0.000) and spreads its attention over itself (0.349) and the previous word (0.197).

WHY CHOOSE THIS?

The shortcut is visible directly.

WHY NOT THIS? (TRADEOFFS)

Seed 0, averaged over heads.

Interview preparation — technical questions

Q: How is the causal mask applied?

A: Scores for keys j > i set to minus infinity before the softmax, so their weights are exactly 0.

Q: What are the three parts of a decoder block?

A: Masked self-attention, cross-attention to the encoder, the FFN; each with add & norm.

Q: In cross-attention, where do Q, K and V come from?

A: Q from the decoder; K and V from the encoder's output.

Q: What did the unmasked model score, both ways?

A: 0.5133 on whole sentences; 0.0877 from the past only (seed 0).

Q: Why can a masked decoder train on whole sentences in one pass?

A: Its outputs equal word-by-word ones (7.11e-15 here).

Interview preparation — theoretical questions

Q: Why does the encoder not need a causal mask?

A: It reads a complete input; nothing in it is being predicted.

Q: What goes wrong without the mask?

A: The model learns to read the answer one position ahead (0.536 of its attention) and fails when it is not there.

Q: Why compare with counting baselines?

A: Accuracy alone is misleading: always predicting '<unk>' scores 0.1853 here.

Q: Why choose the epoch by past-only validation?

A: It scores the model the way it will be used; a whole-sentence score would reward peeking.

Key takeaway and what's next

KEY TAKEAWAY

The causal mask is what makes a decoder honest: without it, a next-word model learns to read the answer one position ahead, looks good when scored like it was trained, and fails when it has to predict from the past. With it, one pass over a sentence equals one pass per word, so training is cheap and matches use.

- Mask: 0.0e+00 weight on future positions; earlier outputs unchanged by a later word (0.00e+00).
- No mask: whole sentence 0.5133 / 0.5167 / 0.5007; past only 0.0877 / 0.1222 / 0.1102.
- Mask: 0.2254 / 0.2442 / 0.2326 either way; bigram table 0.2519; always '<unk>' 0.1853.
- Unmasked attention one position ahead: 0.536; masked: 0.000.

Suggested watch order, if starting from zero

- 1. Masking the future in self-attention (Herman Kamper) — the mask in a few minutes.
- 2. Transformers, Part 7: Decoder (2), masked self-attention (Lennart Svensson) — the mask inside the decoder.

Day 81 preview

- **A full Transformer forward pass.** Encoder blocks read the input, decoder blocks write the output while reading the encoder through cross-attention, on a toy copy task.
- Today's exercise script, `day80_exercise.py`, is causal self-attention and cross-attention on their own: the fourth part of Day 83's Transformer.